



Street Lamp Monitoring and Control System

Student Name : mohamed mostafa kammel hassan elwaly

Student ID : 221017

Student Name : mazen wael

Student ID : 221017

Student Name : mohamed ashraf mohamed Mahmoud

Student ID : 221017597

1. Overview

This project implements a **smart street lamp monitoring system** using an **ESP32 microcontroller** connected via **Wi-Fi** to a **remote MQTT broker**. The system uses **LDR sensors** to adjust brightness and detect lamp failure, and a **Flask web server** provides real-time monitoring with basic access control.

The architecture is composed of two main components:

- **ESP32 Firmware (C++ using Arduino Core)**: Handles sensor readings, brightness control, and publishes system status to MQTT topics.
 - **Web Dashboard (Python using Flask + MQTT)**: Authenticated interface that visualizes lamp status and message logs in real-time using MQTT messages.
-

2. ESP32 Firmware Details

2.1 Connectivity

- **Wi-Fi**: Connects to a local Wi-Fi network (ssid = "Bruh", password = "7a4ee42005").
- **MQTT Broker**: Connects to the broker at IP 102.46.132.3 using port 1883.

2.2 Sensors and Actuators

- **LDR Sensor (Analog)**: Connected to pin 34. Used to measure ambient light and adjust LED brightness accordingly.
- **Bulb Check Sensors (Digital)**: Pins 27, 14, and 13 connected to separate digital LDRs that monitor light from three individual lamps.
- **LED Outputs**: Pins 2, 5, and 4 used as PWM outputs to control brightness of three LEDs representing lamps.

2.3 MQTT Topics

- Publishes **brightness data** to esp32/bruh/ldr
- Publishes **lamp status** to esp32/bruh/status
- Receives commands (future support suggested) from esp32/bruh/control

2.4 Logic

- Reads LDR value every cycle, maps it to brightness (0–255), and adjusts LED brightness.
 - Every 5 seconds, checks if a lamp is *"broken"* (i.e., high brightness expected but no light detected) and publishes its status.
-

3. Flask Web Dashboard

3.1 Technologies Used

- **Flask** for web routing and rendering
- **paho-mqtt** for MQTT communication
- **HTTP Basic Authentication** via Flask-HTTPAuth
- **Thread-safe logs** using threading.Lock

3.2 MQTT Integration

- Subscribes to esp32/bruh/status
- Publishes control commands to esp32/bruh/control (future use)

3.3 Features

- Real-time visualization of **lamp status** (OK or Broken)
 - **Timestamped log** of messages received from the ESP32
 - **Auto-refresh** every 5 seconds for live updates
 - **Reset buttons** to acknowledge and clear error states
 - **Basic authentication** to restrict access (admin/ChangeThisPassword123!)
-

Step	Device	Description
1	ESP32	Connects to Wi-Fi and MQTT broker.
2	ESP32	Reads LDR and digital bulbs; publishes brightness and status.
3	Flask App	Subscribes to esp32/bruh/status, parses messages.
4	Flask App	Updates internal state (lamp_status) based on messages.
5	User	Views live lamp status on dashboard.
6	User	Clicks "Reset" to mark a lamp as "OK" manually.

6. Recommendations for Improvement

- **Add QoS and LWT:** Improve MQTT reliability and detect disconnection.
 - **Enhance Security:**
 - Replace hardcoded Wi-Fi/MQTT credentials with secure storage.
 - Use SSL for MQTT and HTTPS for Flask.
 - **Add Control Functionality:**
 - Use esp32/bruh/control to allow the server to turn lamps ON/OFF remotely.
 - **Make Lamp Reset Bi-directional:**
 - When "Reset" is clicked, send MQTT control message to ESP32 to recheck state.
 - **Use Non-blocking Delay (millis):**
 - Replace delay(50) with time checks to avoid blocking the main loop.
-

7. Conclusion

This integrated system demonstrates a **robust IoT solution** for street lamp monitoring using **ESP32 and MQTT**, backed by a **Flask dashboard** for real-time visualization. It successfully detects lamp failure, logs events, and provides a user interface for monitoring and management.

This architecture can be extended for large-scale deployments in smart cities or industrial environments.